```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score

train='/content/fraudTrain.csv'
test='/content/fraudTest.csv'

train_dataset =  pd.read_csv(train, encoding='latin-1')
test_dataset =  pd.read_csv(test, encoding='latin-1')

train_dataset.head()
```

```
{"type":"dataframe","variable_name":"train_dataset"}
```

```
test_dataset.head()
```

```
{"type":"dataframe","variable_name":"test_dataset"}
```

```
#Count the number of fraud for train data...
train_dataset['is_fraud'].value_counts()
```

```
is_fraud
0.0    73216
1.0      707
Name: count, dtype: int64
```

```
#Count the number of fraud for test data...
test_dataset['is_fraud'].value_counts()
```

```
is_fraud
0.0    81353
1.0      306
Name: count, dtype: int64
```

```
# Check any null value is remain or not in train data...
train_dataset.isnull().sum()
```

```
Unnamed: 0                0
trans_date_trans_time     0
cc_num                    0
merchant                  0
category                  0
amt                       0
first                     0
last                      0
gender                    0
street                    0
city                      1
```

```
state                     1
zip                       1
lat                       1
long                      1
city_pop                  1
job                       1
dob                       1
trans_num                 1
unix_time                 1
merch_lat                 1
merch_long                1
is_fraud                  1
dtype: int64
```

```
# Check any null value is remain or not in test data...
test_dataset.isnull().sum()
```

```
Unnamed: 0                0
trans_date_trans_time     0
cc_num                    0
merchant                  1
category                  1
amt                       1
first                     1
last                      1
gender                    1
street                    1
city                      1
state                     1
zip                       1
lat                       1
long                      1
city_pop                  1
job                       1
dob                       1
trans_num                 1
unix_time                 1
merch_lat                 1
merch_long                1
is_fraud                  1
dtype: int64
```

```
#Find non-numerical data in train dataset
train_dataset.select_dtypes(exclude='number').columns
```

```
Index(['trans_date_trans_time', 'merchant', 'category', 'first',
'last',
       'gender', 'street', 'city', 'state', 'job', 'dob',
'trans_num'],
      dtype='object')
```

```
#Find non-numerical data in test dataset
test_dataset.select_dtypes(exclude='number').columns
```

```
Index(['trans_date_trans_time', 'merchant', 'category', 'first',
'last',
       'gender', 'street', 'city', 'state', 'job', 'dob',
'trans_num'],
      dtype='object')
```

```
from sklearn.preprocessing import LabelEncoder
label_encoders = {}

label_encode_cols = ['merchant', 'category', 'gender', 'state', 'job']
for col in label_encode_cols:
    le = LabelEncoder()

    train_dataset[col] = le.fit_transform(train_dataset[col])
    label_encoders[col] = le

    test_dataset[col] = le.fit_transform(test_dataset[col])
    label_encoders[col] = le

train_dataset.drop(['first', 'last', 'street', 'city', 'trans_num'],
axis=1, inplace=True)
test_dataset.drop(['first', 'last', 'street', 'city', 'trans_num'],
axis=1, inplace=True)

# Convert columns to datetime
train_dataset['trans_date_trans_time'] =
pd.to_datetime(train_dataset['trans_date_trans_time'])
train_dataset['dob'] = pd.to_datetime(train_dataset['dob'])

# Extract year, month, day, and hour for the transaction date
train_dataset['transaction_year'] =
train_dataset['trans_date_trans_time'].dt.year
train_dataset['transaction_month'] =
train_dataset['trans_date_trans_time'].dt.month
train_dataset['transaction_day'] =
train_dataset['trans_date_trans_time'].dt.day
train_dataset['transaction_hour'] =
train_dataset['trans_date_trans_time'].dt.hour

train_dataset['birth_year'] = train_dataset['dob'].dt.year
train_dataset['birth_month'] = train_dataset['dob'].dt.month
train_dataset['birth_day'] = train_dataset['dob'].dt.day

train_dataset.drop(['trans_date_trans_time', 'dob'], axis=1,
inplace=True)


#Test part
```

```
# Convert columns to datetime
test_dataset['trans_date_trans_time'] =
pd.to_datetime(test_dataset['trans_date_trans_time'])
test_dataset['dob'] = pd.to_datetime(test_dataset['dob'])

# Extract year, month, day, and hour for the transaction date
test_dataset['transaction_year'] =
test_dataset['trans_date_trans_time'].dt.year
test_dataset['transaction_month'] =
test_dataset['trans_date_trans_time'].dt.month
test_dataset['transaction_day'] =
test_dataset['trans_date_trans_time'].dt.day
test_dataset['transaction_hour'] =
test_dataset['trans_date_trans_time'].dt.hour

test_dataset['birth_year'] = test_dataset['dob'].dt.year
test_dataset['birth_month'] = test_dataset['dob'].dt.month
test_dataset['birth_day'] = test_dataset['dob'].dt.day

test_dataset.drop(['trans_date_trans_time', 'dob'], axis=1,
inplace=True)

print(train_dataset.shape)
print(test_dataset.shape)
```

```
(73924, 23)
(81660, 23)
```

```
print(train_dataset.head(0))
print(train_dataset.head())

print(test_dataset.head(0))
print(test_dataset.head())
```

```
Empty DataFrame
Columns: [Unnamed: 0, cc_num, merchant, category, amt, gender, state,
zip, lat, long, city_pop, job, unix_time, merch_lat, merch_long,
is_fraud, transaction_year, transaction_month, transaction_day,
transaction_hour, birth_year, birth_month, birth_day]
Index: []

[0 rows x 23 columns]
   Unnamed: 0            cc_num  merchant  category     amt  gender
state  \
0           0  2703186189652095       514         8    4.97       0
26
1           1      630423337322       241         4  107.23       0
46
2           2    38859492057661       390         0  220.11       1
12
```

```
3           3  3534093764340240       360         2   45.00       1
25
4           4   375534208663984       297         9   41.96       1
44

       zip      lat      long  ...  merch_lat  merch_long  is_fraud  \
0  28654.0  36.0788  -81.1781  ...  36.011293  -82.048315       0.0
1  99160.0  48.8878 -118.2105  ...  49.159047 -118.186462       0.0
2  83252.0  42.1808 -112.2620  ...  43.150704 -112.154481       0.0
3  59632.0  46.2306 -112.1138  ...  47.034331 -112.561071       0.0
4  24433.0  38.4207  -79.4629  ...  38.674999  -78.632459       0.0

   transaction_year  transaction_month  transaction_day
transaction_hour  \
0              2019                  1                1
0
1              2019                  1                1
0
2              2019                  1                1
0
3              2019                  1                1
0
4              2019                  1                1
0

   birth_year  birth_month  birth_day
0      1988.0          3.0        9.0
1      1978.0          6.0       21.0
2      1962.0          1.0       19.0
3      1967.0          1.0       12.0
4      1986.0          3.0       28.0

[5 rows x 23 columns]
Empty DataFrame
Columns: [Unnamed: 0, cc_num, merchant, category, amt, gender, state,
zip, lat, long, city_pop, job, unix_time, merch_lat, merch_long,
is_fraud, transaction_year, transaction_month, transaction_day,
transaction_hour, birth_year, birth_month, birth_day]
Index: []

[0 rows x 23 columns]
   Unnamed: 0            cc_num  merchant  category    amt  gender
state  \
0           0  2291163933867244       319        10   2.86       1
39
1           1  3573030041201292       591        10  29.84       0
43
2           2  3598215285024754       611         5  41.28       0
33
3           3  3591919803438423       222         9  60.05       1
```

```
8
4           4  3526826139003047       292        13   3.19       1
21

       zip      lat      long  ...  merch_lat  merch_long  is_fraud  \
0  29209.0  33.9659  -80.9355  ...  33.986391  -81.200714       0.0
1  84002.0  40.3207 -110.4360  ...  39.450498 -109.960431       0.0
2  11710.0  40.6729  -73.5365  ...  40.495810  -74.196111       0.0
3  32780.0  28.5697  -80.8191  ...  28.812398  -80.883061       0.0
4  49632.0  44.2529  -85.0170  ...  44.959148  -85.884734       0.0

   transaction_year  transaction_month  transaction_day
transaction_hour  \
0              2020                  6               21
12
1              2020                  6               21
12
2              2020                  6               21
12
3              2020                  6               21
12
4              2020                  6               21
12

   birth_year  birth_month  birth_day
0      1968.0          3.0       19.0
1      1990.0          1.0       17.0
2      1970.0         10.0       21.0
3      1987.0          7.0       25.0
4      1955.0          7.0        6.0

[5 rows x 23 columns]

print(train_dataset.shape)
print(test_dataset.shape)

(73924, 23)
(81660, 23)

# Apply one-hot encoding to the training dataset
df_processed = pd.get_dummies(train_dataset)
df_processed

{"type":"dataframe","variable_name":"df_processed"}

# Extract features and target variable from the processed dataframe
x_train = df_processed.drop(columns='is_fraud')
y_train = df_processed['is_fraud']

# Drop rows with NaN values from x_train and y_train
# This ensures no NaNs are passed to the model during training.
```

```
combined_train = pd.concat([x_train, y_train], axis=1).dropna()
x_train = combined_train.drop(columns='is_fraud')
y_train = combined_train['is_fraud']

df_processed_test = pd.get_dummies(data=test_dataset)
df_processed_test
```

```
{"type":"dataframe","variable_name":"df_processed_test"}
```

```
x_test = df_processed_test.drop(columns='is_fraud', axis=1)
y_test = df_processed_test['is_fraud']
```

Training and Testing the LogisticRegression

```
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
# Initialize the classifiers
from sklearn.linear_model import LogisticRegression
logistic_regression = LogisticRegression()

# Train the classifiers
logistic_regression.fit(x_train, y_train)

# Make predictions
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
y_pred_lr = logistic_regression.predict(x_test)

# Evaluate the models
print("Logistic Regression Accuracy:", accuracy_score(y_test,
y_pred_lr))
print("Logistic Regression Classification Report:\n",
classification_report(y_test, y_pred_lr))
print("Logistic Regression Confusion Matrix:\n",
confusion_matrix(y_test, y_pred_lr))
```

```
Logistic Regression Accuracy: 0.9962527094380289
Logistic Regression Classification Report:
               precision    recall  f1-score   support

         0.0       1.00      1.00      1.00     81353
         1.0       0.00      0.00      0.00       306

    accuracy                           1.00     81659
   macro avg       0.50      0.50      0.50     81659
weighted avg       0.99      1.00      0.99     81659

Logistic Regression Confusion Matrix:
 [[81353     0]
 [  306     0]]
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatio
n.py:1565: UndefinedMetricWarning: Precision is ill-defined and being
set to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatio
n.py:1565: UndefinedMetricWarning: Precision is ill-defined and being
set to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
```

Training and Testing the Decision Trees

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Initialize the Decision Tree classifier
decision_tree = DecisionTreeClassifier()

# Train the Decision Tree classifier
decision_tree.fit(x_train, y_train)

# Make predictions
y_pred_dt = decision_tree.predict(x_test)

# Evaluate the Decision Tree model
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Decision Tree Classification Report:\n",
classification_report(y_test, y_pred_dt))
print("Decision Tree Confusion Matrix:\n", confusion_matrix(y_test,
y_pred_dt))
```

```
Decision Tree Accuracy: 0.9958608359152084
Decision Tree Classification Report:
              precision    recall  f1-score   support

         0.0       1.00      1.00      1.00     81353
         1.0       0.46      0.68      0.55       306

    accuracy                           1.00     81659
   macro avg       0.73      0.84      0.78     81659
```

```
weighted avg       1.00      1.00      1.00     81659

Decision Tree Confusion Matrix:
 [[81112   241]
 [   97   209]]
```

Training and Testing the Random Forests

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Initialize the Random Forest classifier
random_forest = RandomForestClassifier(n_estimators=100,
random_state=42)

# Train the classifier
random_forest.fit(x_train, y_train)

# Make predictions
y_pred_rf = random_forest.predict(x_test)

# Evaluate the model
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print("Random Forest Classification Report:\n",
classification_report(y_test, y_pred_rf))
print("Random Forest Confusion Matrix:\n", confusion_matrix(y_test,
y_pred_rf))
```

```
Random Forest Accuracy: 0.9977222351486058
Random Forest Classification Report:
              precision    recall  f1-score   support

         0.0       1.00      1.00      1.00     81353
         1.0       0.93      0.42      0.58       306

    accuracy                           1.00     81659
   macro avg       0.97      0.71      0.79     81659
weighted avg       1.00      1.00      1.00     81659

Random Forest Confusion Matrix:
 [[81344     9]
 [  177   129]]
```